

Public Key Cryptography

Public Key Cryptography

Kerckhoff's Principle: the cryptosystem (algorithm) is public

Symmetric or shared key: only the key is secret, used both for encryption and decryption. Shared keys for MACs, PRFs, etc..

Asymmetric cryptographic capability: encryption capability without decryption capability.

Public Key Cryptosystem (PKC)

Kerckhoff: cryptosystem (algorithm) is public

DH76: can encryption key be public too?

- Decryption key will be different (and private)
- Everybody can send me emails, but only I can read them.

Is it Only About Encryption?

Also: digital signatures for integrity and non-repudiation.

Sign with private key s , verify with public key v .

Recall MACs: a shared key cryptosystem for message authentication

Private signing key, public validation key

More: Key-Exchange Protocols

Establish shared key between Alice and Bob without assuming an existing shared (master) key

Use public information from Alice and Bob to setup shared key k

Eavesdroppers cannot learn the key k

Public keys solve more problems

Signatures provide evidence (everyone can validate, only owner can sign)

Established shared secret keys

- Use authenticated public keys, signed by trusted certificate authority
- Or use DH (Diffie Hellman) key exchange

Stronger resiliency to key exposure

- Perfect forward secrecy and recover security
- Threshold security: resilient to key exposure of t out of n parties

Public keys are easier... To distribute: from directory or from incoming message (still require authentication), and less keys to distribute

To maintain: can keep in non-secure storage as long as being validated before using. Also less keys: $O(|parties|)$ versus $O(|parties|^2)$

So why not always use public key crypto?

The Price of PKC

Assumptions: Applied PKC algorithms are based on a small number of specific computational assumptions

- Mainly: hardness of factoring and discrete-log
- Both may fail against quantum computers

Overhead:

- Computational
- Key length (public key is long)
- Output length (e.g. ciphertext or signature)

Public key crypto is harder...

Requires related public, private keys

- Usually we say a keypair (pk, sk)
- Public key does not expose private key

Substantial overhead

- Successful cryptanalytic shortcuts → need long keys
- Elliptic curves (EC) may allow shorter keys (almost no shortcuts found)
- Complex computations, e.g. complex (slow) keygen

In summary

Minimize use of PKC

If possible, minimize length of the input

How?

For signatures, hash-then-sign

For public-key encryption, hybrid encryption

Hybrid Encryption

Challenge: public key cryptosystems are slow

Hybrid encryption:

- Use a shared key encryption scheme to encrypt all messages,
- But use a public key encryption scheme to exchange the shared key
 - Alice generates k , encrypts it under Bob's public key and sends the ciphertext c_k to Bob.
 - Bob can decrypt and recover k , and use k to decrypt c_M

Number Theory Review

$$(a + b) \bmod m = [(a \bmod m) + (b \bmod m)] \bmod m$$

Multiplicative Inverse

To compute $a/c \bmod m$, multiply a by the multiplicative inverse of c

That is, compute $a/c \bmod m = ac^{-1} \bmod m$

Where c^{-1} is the multiplicative inverse such that $cc^{-1} \bmod m = 1$

Not all ints have multiplicative inverses with respect to a specific modulus m .

Modular Exponentiation

Encountered a lot: discrete log scheme, RSA, etc

Fermat's Little Theorem (for prime modulus):

$$a^b \bmod p = (a \bmod p)^{b \bmod (p-1)} \bmod p$$

Key Exchange

Defining a Key Exchange Protocol

Must satisfy:

Correctness: both parties compute the same shared key

Key Indistinguishability: the key that the two parties establish is indistinguishable from random